Candidat : Jean-François Chaussoy – Data Engineer (alternance)

Entreprise accueillante pour l'alternance : Communauté d'Agglomération du Boulonnais

Date : Avril 2026

Destinataires : Évaluateur & Mentorat

Rapport de Gestion de Projet – Puls-Events

Étude de passage du POC au MVP pour l'assistant conversationnel

Portfolio Web : https://jfc-webcraft.fr/

Profil GitHub : https://github.com/jfcwebcraft

Dépôt GitHub du Projet : jfcwebcraft / Projet-13-Passez-votre-systeme-IA-du-POC-au-MVP

1. Introduction et contexte

1.1 Contexte de Puls-Events

1.2 Bilan du POC (Projet 11)

1.3 Objectifs de l'étude vers le MVP

1.4 Limites techniques du POC à lever

2. Analyse et synthèse des besoins

2.1 Besoins fonctionnels

2.2 Exigences non-fonctionnelles

2.3 Cas d'usage cibles (Personas)

2.4 Parties prenantes du projet

3. Veille technologique et choix de l'hébergeur Cloud

3.1 Comparatif des fournisseurs Cloud

3.2 Justification de l'orientation vers Microsoft Azure

4. Architecture cible proposée pour le MVP

4.1 Découpage en trois briques applicatives

5. Macro backlog des fonctionnalités

5.1 Priorisation du Backlog (Méthode MoSCoW)

6. Estimation des coûts

6.1 Coûts de réalisation (BUILD)

6.2 Coûts d'exploitation mensuels (OPEX)

6.3 Hypothèses de calcul et leviers de maîtrise des coûts

7. Plan de projet et jalons

7.1 Planning prévisionnel sur 8 semaines

7.2 Jalons de validation (Milestones)

8. Gestion des risques

9. Méthodologie de gestion de projet

10. Bilan et perspectives d'évolution

10.1 Synthèse de l'étude

10.2 Évolutions envisageables après le MVP

11. Annexes – Parcours et Portfolio Data Engineer

11.1 Contexte du parcours professionnel

11.2 Récapitulatif des projets réalisés

1. Introduction et contexte

1.1 Contexte de Puls-Events

Puls-Events développe une plateforme web dédiée au référencement et à la découverte d'événements culturels (concerts, expositions, spectacles, festivals). Le service s'appuie principalement sur l'agrégation de données ouvertes issues de l'API publique OpenAgenda afin de proposer des recommandations filtrées selon la localisation, les disponibilités et les préférences des utilisateurs.

Face au volume important d'événements et à la mise à jour fréquente des programmations, la recherche classique par mots-clés et filtres stricts montre ses limites dès qu'un utilisateur cherche une idée de sortie avec des critères croisés. L'objectif est donc d'intégrer un moteur de recherche conversationnel basé sur du traitement du langage naturel (NLP) et une architecture RAG (Retrieval-Augmented Generation), capable d'interpréter une demande formulée librement.

1.2 Bilan du POC (Projet 11)

Lors du Projet 11, un premier démonstrateur technique (POC) a été développé en local pour tester la faisabilité d'une chaîne RAG sur les données d'OpenAgenda.

Ce prototype reposait sur un périmètre restreint : une extraction ponctuelle des événements d'Île-de-France sur les douze derniers mois, soit 488 événements bruts ramenés à 485 après nettoyage et déduplication avec Pandas. Les textes ont été découpés en 495 segments (chunks), vectorisés en dimension 1024 via le modèle mistral-embed, puis stockés dans un index FAISS local exécuté sur CPU (IndexFlatL2). L'orchestration des requêtes vers le modèle mistral-small-latest était assurée par LangChain, l'ensemble étant piloté via un simple script en ligne de commande (chatbot.py).

Les tests réalisés sur un jeu de cinq questions (qa_dataset.json) ont affiché une similarité cosinus moyenne de 89,6 % entre les réponses générées et les réponses attendues. Si ce résultat valide le principe technique sur un échantillon réduit, il ne permet pas de préjuger de la pertinence du classement (Top-K) ni du comportement du système face à une charge réelle.

1.3 Objectifs de l'étude vers le MVP

Ce document présente l'étude de cadrage visant à transformer ce script local en un Produit Minimum Viable (MVP) déployable sur le cloud. Il détaille l'architecture cible envisagée, la priorisation du backlog, le chiffrage prévisionnel (coûts de développement et d'exploitation) ainsi que le planning de réalisation sur huit semaines. L'ensemble de ces propositions techniques et budgétaires constitue une base de travail soumise à la validation de la Direction Technique avant le lancement des développements.

1.4 Limites techniques du POC à lever

Limite actuelle du POC	Impact pour une mise en production
Exécution en console locale	Le script s'exécute dans une boucle locale mono-utilisateur ; aucune API web ne permet de gérer des requêtes simultanées.
Index FAISS sur disque local	Les vecteurs et métadonnées sont liés au poste de développement, sans mécanisme de partage entre plusieurs conteneurs.
Absence de mémoire de session	Chaque question est traitée de manière isolée ; le système oublie le contexte dès la question suivante.
Extraction de données manuelle	Aucune tâche planifiée ne met à jour le catalogue ; la base devient rapidement obsolète.
Aucune télémétrie	Impossible de suivre le temps de réponse (P95), le taux d'erreur ou la consommation de jetons API.

2. Analyse et synthèse des besoins

2.1 Besoins fonctionnels

Fonctionnalité	Description	Priorité (MoSCoW)
Interface de chat	Dialogue en langage naturel avec réponses rédigées en français.	Must-Have
Recherche sémantique (RAG)	Croisement entre la demande de l'utilisateur et le catalogue d'événements indexés.	Must-Have
Mémoire conversationnelle	Conservation des derniers échanges de la session (fenêtre glissante) pour les questions de suivi.	Must-Have
Supervision et métriques	Suivi de la disponibilité de l'API, de la latence et de la consommation LLM.	Should-Have
Filtre géographique	Prise en compte de la ville ou d'un rayon kilométrique pour trier les résultats.	Could-Have
Recherche web d'appoint	Requête web complémentaire lorsque l'événement demandé est absent de la base locale.	Could-Have

2.2 Exigences non-fonctionnelles

Critère	Cible visée pour le MVP
Scalabilité	Déploiement conteneurisé avec mise à l'échelle automatique (0 à N instances) sur Azure Container Apps.
Temps de réponse	Latence globale inférieure à 5 secondes au 95e percentile (P95), à confirmer lors des tests de charge.
Disponibilité	Objectif de 99,5 % sur les plages d'utilisation principales.
Sécurité	Flux chiffrés en TLS et stockage des clés d'API tierces dans Azure Key Vault.
Qualité du code	Code Python typé (Type Hints), validé par linter et couvert par des tests unitaires pytest.

2.3 Cas d'usage cibles (Personas)

Lucas, 28 ans – Résident local cherchant une sortie de dernière minute

Lucas habite en centre-ville et cherche régulièrement des idées de sorties pour le week-end à proximité de chez lui. Lorsqu'il demande : « Quels sont les concerts gratuits ce samedi soir à moins de 5 km ? », le système doit extraire l'intention (concert gratuit), la contrainte temporelle (samedi soir) et le périmètre géographique. Le moteur interroge la base vectorielle en filtrant les événements passés, pondère les résultats selon la distance et renvoie une sélection concise accompagnée des liens officiels.

Émilie, 35 ans – En court séjour touristique

En visite pour trois jours dans une ville qu'elle ne connaît pas, Émilie utilise l'assistant pour organiser son programme. Elle commence par demander : « Je cherche une exposition d'art contemporain ouverte vendredi après-midi ». Une fois les premières pistes affichées, elle poursuit avec : « Est-ce que la deuxième est adaptée pour des enfants ? ». Grâce à la mémoire de session, l'assistant identifie l'exposition concernée et lui répond sans qu'elle ait besoin de reformuler toute sa recherche.

2.4 Parties prenantes du projet

Acteur	Attentes principales	Suivi prévu
Équipe Produit	Tenue des délais, pertinence des réponses et fiabilité des informations pratiques affichées.	Points d'avancement et démonstrations toutes les deux semaines.
Équipe Marketing	Qualité de l'expérience utilisateur et mise en valeur des sources partenaires.	Suivi des retours utilisateurs et vérification du format des réponses.
Direction Technique	Architecture maintenable, tests automatisés et maîtrise des coûts cloud.	Revues de code (Pull Requests) et validation des choix d'infrastructure.
Utilisateurs finaux	Réponses rapides (< 5 s), exactes et accompagnées de liens fonctionnels.	Campagne de bêta-test et suivi des temps de réponse.

3. Veille technologique et choix de l'hébergeur Cloud

3.1 Comparatif des fournisseurs Cloud

Critère	AWS	Microsoft Azure (Retenu)	Google Cloud (GCP)
Exécution de conteneurs	ECS / Fargate (robuste, mais configuration réseau et IAM plus lourde)	**Azure Container Apps** (serverless, autoscaling KEDA intégré, facturation à l'usage)	Cloud Run (très rapide au démarrage, simple à déployer)
Base SQL managée	RDS PostgreSQL (pas de mise en pause complète native sur les instances standards)	**Azure SQL Database** (mode Serverless avec mise en pause automatique)	Cloud SQL (facturation continue tant que l'instance tourne)
Supervision & Logs	CloudWatch (complet mais tarification éclatée par métrique/log)	**Azure Monitor & App Insights** (intégration native simple avec FastAPI)	Cloud Monitoring & Logging (bien intégré à Cloud Run)
Écosystème IA / LLM	Amazon Bedrock	**Azure AI Services** & partenariat étroit avec Mistral AI	Vertex AI (orienté Gemini)
Localisation des données	Région Paris (eu-west-3)	**Région France Central** (Paris)	Région Paris (europe-west9)

3.2 Justification de l'orientation vers Microsoft Azure

Pour une équipe réduite et un service dont le trafic initial sera variable, Microsoft Azure offre un compromis intéressant entre simplicité d'exploitation et maîtrise des coûts fixes.

L'utilisation combinée d'Azure Container Apps et d'Azure SQL Database en mode Serverless permet de réduire la facturation lors des périodes d'inactivité (la nuit ou en creux de semaine), sous réserve de bien paramétrer les délais de mise en veille et de gérer le temps de réveil (cold start) côté applicatif. Par ailleurs, l'utilisation de conteneurs Docker standards limite l'adhérence au fournisseur : le cœur applicatif (FastAPI

et pipeline d'ingestion) reste portable vers un autre hébergeur si nécessaire.

Documentation de référence consultée pour le dimensionnement :

- Facturation Azure Container Apps : https://learn.microsoft.com/azure/container-apps/billing
- Mise à l'échelle Azure Container Apps : https://learn.microsoft.com/azure/container-apps/scale-app
- Niveau Serverless Azure SQL Database : https://learn.microsoft.com/en-us/azure/azure-sql/database/serverless-tier-overview?view=azuresql

4. Architecture cible proposée pour le MVP

4.1 Découpage en trois briques applicatives

1. Pipeline d'acquisition et d'indexation des données

La mise à jour du catalogue reposera sur une tâche planifiée (Job Container Apps) exécutant un script Python d'interrogation de l'API OpenAgenda. Après nettoyage et déduplication des entrées via Pandas, les métadonnées structurées (identifiants, dates, horaires, coordonnées GPS, tarifs) seront enregistrées dans Azure SQL Database. En parallèle, les titres et descriptions seront vectorisés via l'API mistral-embed pour mettre à jour l'index FAISS. Ce dernier sera stocké sur Azure Blob Storage afin que les conteneurs de l'API puissent le charger en mémoire au démarrage ou lors des rafraîchissements.

2. Cœur applicatif et orchestration RAG

Une API REST développée avec FastAPI servira de point d'entrée unique. À chaque requête, elle vérifiera d'abord si une question similaire figure dans le cache sémantique. Dans le cas contraire, elle interrogera l'index FAISS tout en filtrant les événements dont la date est dépassée via les métadonnées SQL. L'application sera instrumentée avec Azure Application Insights pour suivre les temps de réponse de chaque brique et surveiller les coûts d'appels vers Mistral AI.

3. Gestion du dialogue et restitution utilisateur

Pour permettre des échanges naturels sur plusieurs messages, l'API conservera un historique glissant des derniers tours de parole associé à l'identifiant de session. Si l'utilisateur partage sa ville ou sa position, ce

paramètre sera utilisé pour pondérer le tri des événements proches. Enfin, le prompt système imposera au modèle mistral-small de s'appuyer exclusivement sur les événements transmis dans le contexte et d'afficher les liens sources afin de limiter les risques d'invention (hallucinations).

5. Macro backlog des fonctionnalités

5.1 Priorisation du Backlog (Méthode MoSCoW)

Fonctionnalité	Priorité	Complexité	Charge	Risque principal	Parade prévue
Connecteur OpenAgenda & tâche planifiée	**Must**	Moyenne	3 j	Limites de requêtes API	Pagination et retry avec délai exponentiel
Nettoyage et déduplication des données	**Must**	Moyenne	2 j	Champs manquants ou mal formés	Schémas de validation Pydantic stricts
Persistance SQL des métadonnées	**Must**	Moyenne	2 j	Évolution de la structure	Migrations de schéma versionnées
Génération des embeddings & index FAISS	**Must**	Moyenne	4 j	Coût et lenteur d'indexation	Appels groupés (batching) uniquement sur les nouveautés
API FastAPI (endpoints recherche & chat)	**Must**	Élevée	4 j	Code monolithique difficile à tester	Découpage par routeurs et injection de dépendances

Fonctionnalité	Priorité	Complexité	Charge	Risque principal	Parade prévue
Chaîne RAG avec citation des sources	**Must**	Élevée	4 j	Invention d'événements	Prompt contraint et vérification des liens retournés
Mémoire conversationnelle	**Must**	Faible	2 j	Saturation de la fenêtre de contexte	Fenêtre glissante limitée aux derniers échanges
Conteneurisation Docker & pipeline CI	**Must**	Faible	2 j	Écarts entre local et cloud	Tests et lint bloquants dans GitHub Actions
Déploiement Azure (Container Apps + SQL)	**Must**	Moyenne	3 j	Modifications manuelles non tracées	Déploiement scripté et revu sur dépôt Git
Cache sémantique des réponses	**Should**	Moyenne	2 j	Renvoi d'informations périmées	Durée de vie (TTL) courte et purge après ingestion
Télémétrie Application Insights	**Should**	Faible	1 j	Volume de logs trop coûteux	Échantillonnage (sampling) des traces
Filtre et tri géographique	**Could**	Faible	1 j	Événements sans coordonnées GPS	Repli sur le centre-ville de la commune
Recherche web d'appoint (smolagents)	**Could**	Moyenne	1 j	Sources web peu fiables	Restriction à une liste blanche de sites

Fonctionnalité	Priorité	Complexité	Charge	Risque principal	Parade prévue
					culturels

Volume global estimé : 13 fonctionnalités totalisant **31 jours-hommes** (26 j sur les Must, 3 j sur les Should et 2 j sur les Could).

6. Estimation des coûts

6.1 Coûts de réalisation (BUILD)

Lot de travaux	Charge estimée	Coût prévisionnel (€ HT)
Cadrage technique et priorisation	2 jours	920 €
Pipeline d'ingestion et nettoyage	5 jours	2 300 €
Vectorisation et gestion de l'index FAISS	4 jours	1 840 €
Développement de l'API FastAPI	5 jours	2 300 €
Intégration de la chaîne RAG et de la mémoire	4 jours	1 840 €
Conteneurisation Docker et intégration continue	3 jours	1 380 €
Déploiement sur Azure et configuration	3 jours	1 380 €

Lot de travaux	Charge estimée	Coût prévisionnel (€ HT)
Mise en place du monitoring et tests de charge	5 jours	2 300 €
TOTAL BUILD PRÉVISIONNEL	**31 jours-hommes**	**14 260 € HT**

Note : Ce chiffrage repose sur un taux journalier moyen (TJM) interne valorisé à 460 € HT/jour pour donner un ordre de grandeur de l'investissement.

6.2 Coûts d'exploitation mensuels (OPEX)

Poste d'infrastructure	Service retenu	Estimation mensuelle (HT)
Appels LLM & Embeddings	API Mistral AI (base de 5 000 requêtes/mois, cache déduit)	35,00 €
Hébergement applicatif	Azure Container Apps (plan Consumption)	18,00 €
Base de données	Azure SQL Database Serverless (avec auto-pause)	15,00 €
Stockage de l'index	Azure Blob Storage (~200 Mo pour le fichier FAISS)	Inclus (< 1,00 €)
Journaux et métriques	Azure Log Analytics & Application Insights	8,00 €
TOTAL OPEX INDICATIF	Ensemble de l'infrastructure	**~ 76,00 € / mois**

6.3 Hypothèses de calcul et leviers de maîtrise des coûts

Ce budget mensuel d'environ 76 € HT est une projection établie pour un volume initial de 5 000 questions par mois. Il s'appuie sur trois mécanismes d'optimisation qui devront être vérifiés lors de la phase de test :

1. **Le cache sémantique**, avec l'hypothèse d'intercepter environ 35 % des requêtes récurrentes (ex. demandes fréquentes sur le week-end en cours) afin d'éviter des appels facturés vers l'API Mistral.
2. **La mise en veille automatique** d'Azure SQL Serverless et le passage à zéro réplica sur Container Apps la nuit.
3. **Le rafraîchissement incrémental** des embeddings, pour ne vectoriser que les nouveaux événements ou ceux dont le descriptif a changé.

Ces montants devront être affinés à l'issue des tests de charge en fonction de la taille réelle des prompts et du trafic observé.

7. Plan de projet et jalons

7.1 Planning prévisionnel sur 8 semaines

Phase	Contenu des travaux	Durée	Période
Phase 1	Cadrage fonctionnel et validation du périmètre MVP	1 semaine	Semaine 1

Phase	Contenu des travaux	Durée	Période
Phase 2	Spécification de l'architecture et maquettage Azure	1 semaine	Semaine 2
Phase 3	Développement itératif (2 sprints de 2 semaines)	4 semaines	Semaines 3 à 6
Phase 4	Campagne de tests, sécurité et essais de charge	1 semaine	Semaine 7
Phase 5	Déploiement sur l'environnement cible et recette	1 semaine	Semaine 8

7.2 Jalons de validation (Milestones)

- **J1 (Fin S1) – Validation du périmètre :** Arbitrage du backlog avec l'équipe Produit et gel des fonctionnalités Must-Have.
- **J2 (Fin S2) – Validation de l'architecture :** Revue du schéma cible et du budget prévisionnel avec la Direction Technique.
- **J3 (Fin S4 – Sprint 1) – Démo Data & Indexation :** Démonstration du pipeline d'ingestion automatisé, du stockage SQL et de la mise à jour de l'index FAISS.
- **J4 (Fin S6 – Sprint 2) – Démo API & RAG :** Démonstration du dialogue complet avec mémoire de session et citation des sources via FastAPI.
- **J5 (Fin S7) – Bilan de performance :** Analyse des résultats de tests de charge (vérification de la cible P95 < 5 s) et audit de sécurité.
- **J6 (Fin S8) – Recette finale (Go/No-Go) :** Présentation du MVP déployé aux parties prenantes pour décision d'ouverture aux utilisateurs.

8. Gestion des risques

Risque identifié	Probabilité	Impact	Mesures préventives et correctives
Saturation des quotas ou indisponibilité de l'API Mistral	Moyenne	Élevé	Mise en place d'une limitation de débit (rate limiting) par utilisateur sur FastAPI, gestion des erreurs avec retry et réponse dégradée propre en cas de panne.
Hallucinations du modèle (horaires ou lieux erronés)	Moyenne	Élevé	Température basse du modèle, consigne système stricte imposant de citer les données SQL, et évaluation préalable sur un jeu de questions de référence.
Dépassement du budget Cloud / LLM	Faible	Élevé	Configuration d'alertes budgétaires dans Azure Cost Management (seuils à 50 %, 80 % et 100 %) et plafonnement de la consommation mensuelle sur la console Mistral.
Recommandation d'événements déjà passés	Moyenne	Moyen	Filtrage SQL systématique sur le champ date_fin >= date_du_jour avant même d'envoyer les segments au LLM.

9. Méthodologie de gestion de projet

Pour mener ce projet sur huit semaines en équipe réduite, je propose une organisation hybride combinant un cadrage initial court et un développement en Scrum allégé.

Les deux premières semaines sécurisent les fondations (validation du périmètre, choix des services Azure et préparation des environnements). La phase de réalisation (semaines 3 à 6) s'articule ensuite autour de deux sprints de deux semaines. Le premier sprint se concentre sur la donnée (ingestion OpenAgenda, base SQL et indexation FAISS) afin de disposer rapidement d'un corpus réel. Le second sprint est dédié à l'API

FastAPI, à l'orchestration RAG et aux fonctionnalités conversationnelles. Chaque fin de sprint donne lieu à une démonstration fonctionnelle auprès des équipes Produit et Technique pour ajuster les priorités si nécessaire. Enfin, les deux dernières semaines permettent d'isoler les phases de tests de charge, de sécurisation et de recette avant la décision de mise en ligne.

10. Bilan et perspectives d'évolution

10.1 Synthèse de l'étude

Le prototype réalisé lors du Projet 11 a confirmé l'intérêt de la recherche vectorielle pour interroger un catalogue d'événements culturels en langage naturel. La présente étude trace une feuille de route réaliste pour passer de ce script local à une architecture cloud maintenable : découplage entre l'ingestion et l'API, persistance hybride (SQL + FAISS sur Blob Storage), chiffrage de réalisation à 31 jours-hommes et budget d'exploitation maîtrisé autour de 76 € HT/mois au démarrage. La prochaine étape consiste à valider ces arbitrages avec la Direction Technique pour lancer le premier sprint.

10.2 Évolutions envisageables après le MVP

Une fois le MVP mis entre les mains des premiers utilisateurs et les métriques réelles collectées, trois axes d'amélioration pourront être étudiés :

1. **Migration vers une base vectorielle dédiée (ex. Qdrant ou pgvector) :** si le catalogue dépasse plusieurs dizaines de milliers d'événements ou si le rechargement du fichier FAISS en mémoire devient un goulet d'étranglement lors des mises à jour fréquentes.
2. **Industrialisation de l'évaluation RAG :** intégration d'une revue automatisée de la qualité des réponses sur un jeu de tests élargi à chaque modification du prompt ou du modèle.
3. **Comparaison avec des modèles open-weight auto-hébergés :** si le volume de requêtes augmente fortement, étudier le seuil de rentabilité entre l'API Mistral facturée au jeton et une instance GPU dédiée.

11. Annexes – Parcours et Portfolio Data Engineer

11.1 Contexte du parcours professionnel

Ce rapport de cadrage clôt un parcours complet de formation Data Engineer suivi en alternance (13 projets dont 12 projets techniques opérationnels) couvrant l'ensemble de la chaîne de valeur de la donnée : collecte et exploration, conception de bases relationnelles et NoSQL, modélisation prédictive, architectures cloud, traitement distribué Big Data, orchestration de pipelines et intégration de briques d'IA générative.

Les dépôts de code publics, documentations techniques et démonstrateurs sont accessibles en ligne. Note de confidentialité : Conformément aux exigences de sécurité et de confidentialité professionnelles (notamment pour les projets conduits en alternance au sein de la Communauté d'Agglomération du Boulonnais), les dépôts de code des projets d'entreprise restent strictement privés.

- **Portfolio Web :** https://jfc-webcraft.fr/
- **Profil GitHub :** https://github.com/jfcwebcraft
- **Dépôt GitHub du Projet 13 :** jfcwebcraft / Projet-13-Passez-votre-systeme-IA-du-POC-au-MVP

11.2 Récapitulatif des projets réalisés

Projet	Intitulé du projet	Stack technique principale	Compétences mises en œuvre
P2	Analyse exploratoire de données éducatives mondiales (Academy EdTech)	Python, Pandas, Matplotlib, Seaborn	Nettoyage de données, analyse exploratoire (EDA), filtrage statistique et dataviz
P3	Conception et implémentation d'une base relationnelle (Laplace Immo)	PostgreSQL, SQL, Modélisation UML / Relationnelle	Modélisation conceptuelle et logique (MCD/MLD), requêtes SQL et indexation
P4	Audit technique et conformité d'un environnement de données (SuperSmartMarket)	Python, Pandas, RGPD, Contrôles d'intégrité	Audit de données, détection d'anomalies de CA, sécurité et conformité RGPD
P5	Migration de données médicales vers une base NoSQL conteneurisée	MongoDB, Docker Compose, Python, NoSQL	Modélisation orientée documents, conteneurisation Docker et requêtes d'agrégation
P6	Prédiction de la consommation énergétique de bâtiments (Ville	Python, Scikit-learn, XGBoost, Pandas	Feature engineering, modélisation prédictive

Projet	Intitulé du projet	Stack technique principale	Compétences mises en œuvre
	de Seattle)		(régression) et interprétabilité SHAP
P7	Conception et analyse d'une base NoSQL pour l'impact locatif (NosCités)	MongoDB, PyMongo, Polars, Power BI	Pipelines d'agrégation NoSQL, traitement haute performance Polars et dashboard BI
P8	Infrastructure de données météorologiques et pipeline ELT (Forecast 2.0 GreenCoop)	Python, PostgreSQL, Docker, Pytest, ELT	Développement d'un pipeline ELT automatisé, tests unitaires et intégration continue
P9	Modélisation d'une infrastructure Cloud hybride et streaming temps réel (InduTechData)	Apache Spark, PySpark, Redpanda, Cloud Hybride	Architecture cloud hybride, traitement Big Data distribué et streaming événementiel
P10	Industrialisation de la chaîne budgétaire par orchestration (Mission CAB)	Kestra, Docker, Python, SQL	Orchestration de flux par DAGs, automatisation financière et contrôles de qualité
P11	Développement d'un démonstrateur RAG pour événements culturels (POC Puls-Events)	LangChain, FAISS, API Mistral, Python	Ingestion API OpenAgenda, vectorisation, recherche sémantique et prompt engineering
P12	Plateforme d'Architecture de Données Budgétaires Automatisée (Mission CAB)	Python, PostgreSQL, Docker, Power BI, Architecture Système	Gestion de projet d'infrastructure, modélisation en étoile, automatisation et reporting BI
P13	Cadrage du passage d'un POC IA vers un MVP Cloud (Puls-Events)	Azure Container Apps, FastAPI, RAG, FinOps, Docker	Architecture cible cloud, chiffrage FinOps, backlog MoSCoW et gouvernance agile